hashlock.



Security Audit

Vana (Decentralized Data Economy)

September 2025

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	13
Audit Resources	13
Dependencies	13
Severity Definitions	14
Status Definitions	15
Audit Findings	16
Centralisation	22
Conclusion	23
Our Methodology	24
Disclaimers	26
About Hashlock	27

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Vana team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

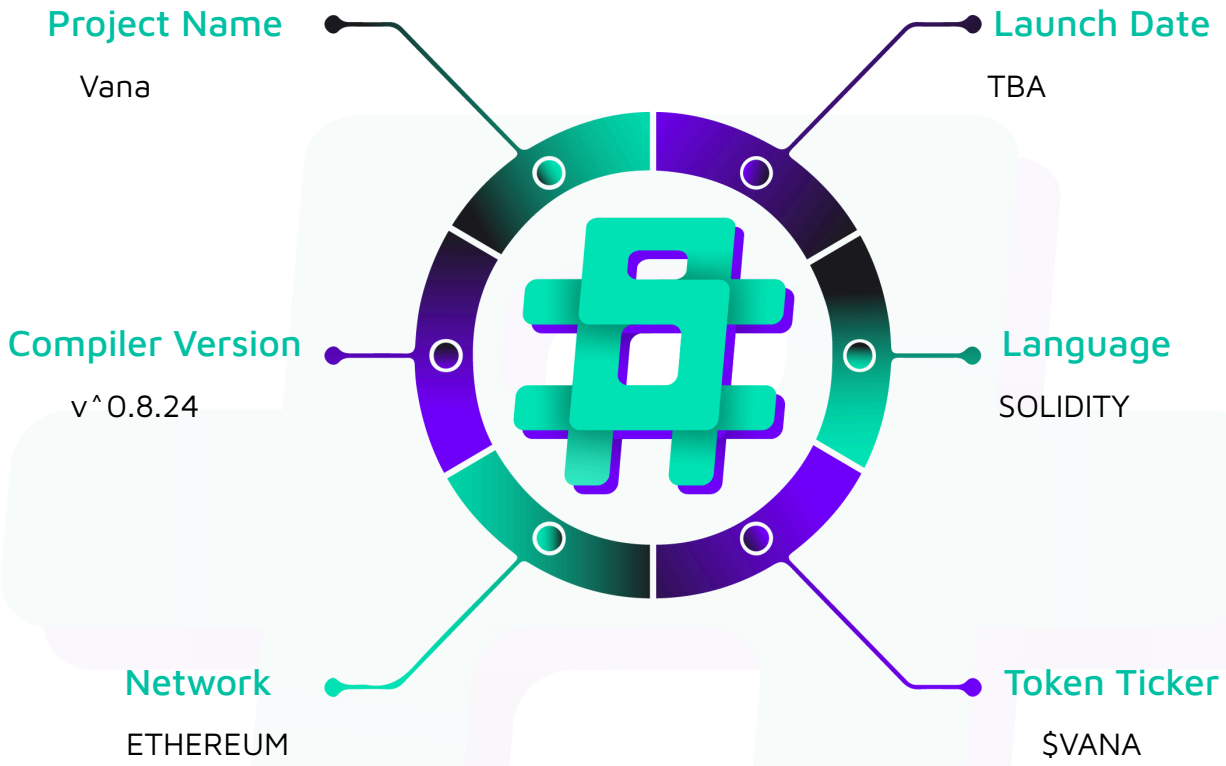
Project Context

Vana is positioning itself as a next-generation data economy platform, focused on user-owned data. It offers tools and services where individuals can contribute their data, launch data collectives, tokenize data assets, and trade data tokens. The core vision is to shift from "data extraction" (where platforms harvest and own your data) to a model where data is owned and controlled by its creators/users, thus enabling participation, value sharing, and transparency.

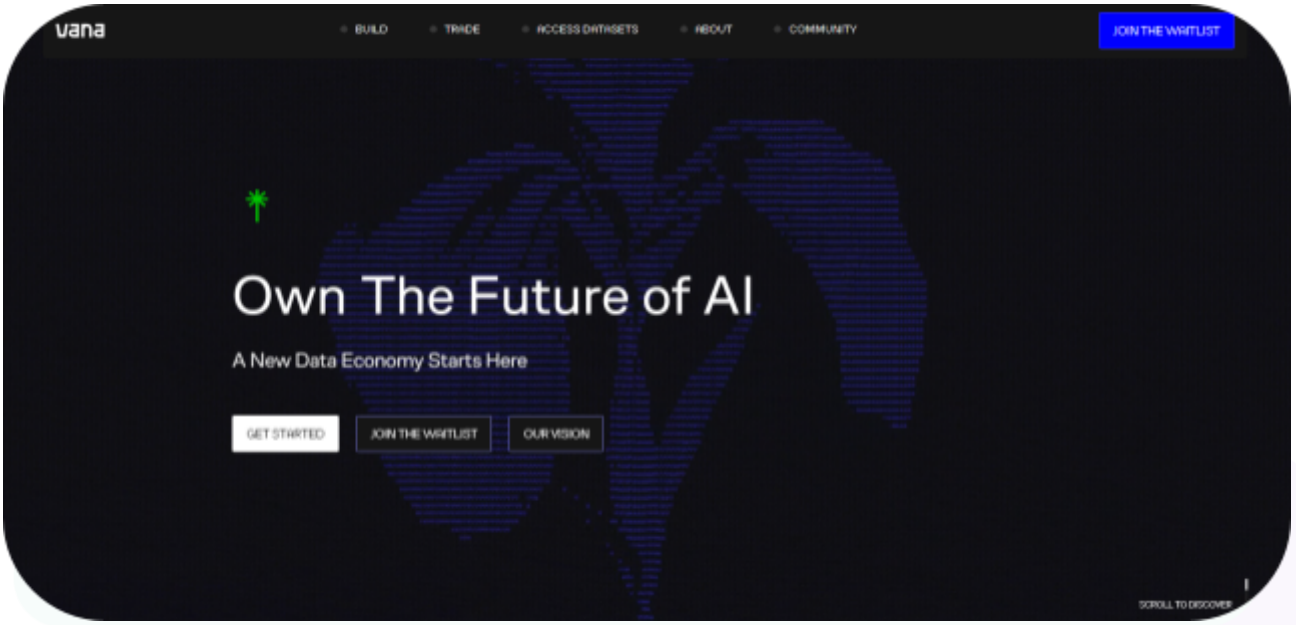
Project Name: Vana
Project Type: Decentralized Data Economy .
Compiler Version: ^0.8.24
Website: https://www.vana.org/
Logo:

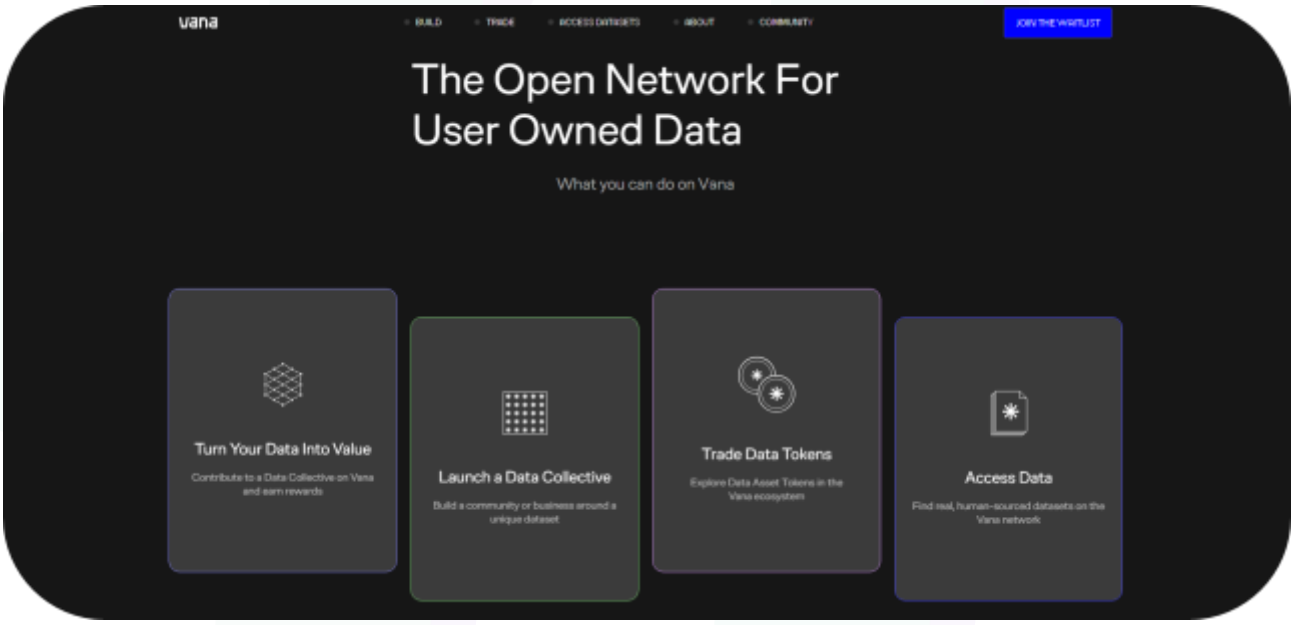


Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Vana project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Vana Smart Contracts
Platform	Ethereum / Solidity
Audit Date	September, 2025
Contract 1	DataPortabilityGranteesImplementation.sol
Contract 2	DataPortabilityGranteesProxy.sol
Contract 3	DataPortabilityGranteesStorageV1.sol
Contract 4	IDataPortabilityGrantees.sol
Contract 5	DataPortabilityPermissionsImplementation.sol
Contract 6	DataPortabilityPermissionsProxy.sol
Contract 7	DataPortabilityPermissionsStorage.sol
Contract 8	IDataPortabilityPermissions.sol
Contract 9	DataPortabilityServersImplementation.sol
Contract 10	DataPortabilityServersProxy.sol
Contract 11	DataPortabilityServersStorageV1.sol
Contract 12	IDataPortabilityServers.sol
Contract 13	DataRefinerRegistryImplementation.sol
Contract 14	DataRefinerRegistryStorageV1.sol
Contract 15	IDataRefinerRegistry.sol
Contract 16	DataRegistryImplementation.sol

Contract 17	DataRegistryStorageV1.sol
Contract 18	IDataRegistry.sol
Contract 19	DLPRegistryImplementation.sol
Contract 20	DLPRewardDeployerImplementation.sol
Contract 21	VanaEpochImplementation.sol
Contract 22	IVanaEpoch.sol
Audited GitHub Commit Hash	b61f20fde7f11eb2f82c20d64f5a0cb0c1eb5449
Fix Review GitHub Commit Hash	18568b915ac553ce93f08c4471101b3d82648d70

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now were resolved or acknowledged.

Hashlock found:

2 Medium severity vulnerabilities

1 Low severity vulnerabilities

3 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
DataPortabilityServersImplementation.sol Allows admins to: - Set role admins - Upgrade the contract Allows maintainers to: - Update trusted forwarder - Pause/unpause the contract - Set user nonces Allows permission managers to: - Add and trust servers for users - Trust servers for users Allows users/owners to: - Update server URLs - Trust/untrust servers (with/without signatures) - Add servers (with signatures) - Add and trust servers (with signatures)	**Contract achieves this functionality.**

DataPortabilityPermissionsImplementation.sol Allows admins to: - Set role admins - Upgrade the contract Allows maintainers to: - Update trusted forwarder - Update data registry, servers, grantees contracts - Pause/unpause the contract Allows users to: - Add permissions (with signatures) - Revoke permissions (with/without signatures) - Add server, files, and permissions (with signatures)	**Contract achieves this functionality.**
DataPortabilityGranteesImplementation.sol Allows admins to: - Set role admins - Upgrade the contract Allows maintainers to: - Update trusted forwarder - Pause/unpause the contract Allows permission managers to:	**Contract achieves this functionality.**

- Add/remove permissions for grantees Allows users to: - Register grantees	

Code Quality

This audit scope involves the smart contracts of the Vana project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Vana project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01]DataPortabilityGranteesImplementation#registerGrantee - Front-Running Vulnerability in Grantee Registration

Description

The `registerGrantee` function is an unprotected external function that allows any address to register a grantee on behalf of an `owner`. It lacks any form of signature verification or authorization check, meaning an attacker can observe a legitimate registration transaction in the mempool and front-run it with their own data.

Impact

An attacker can preemptively register a malicious or incorrect `granteeAddress` and `publicKey` for a legitimate `owner`. Because the contract does not have a mechanism for updating grantee information, this action would be permanent, leading to a persistent denial of service for the legitimate user or the association of a malicious entity with their account.

Recommendation

To prevent unauthorized registrations and front-running, the function should be implementing the `EIP-712` Signature Verification: Require a valid signature from the `owner` address to authorize the registration. This cryptographically proves that the owner initiated the request and prevents third parties from registering on their behalf.

Status

Resolved

[M-02] DataPortabilityContracts - Address Spoofing Risk from Combined MultiCall and ERC2771 Usage

Description

The contract inherits both `MulticallUpgradeable` and `ERC2771ContextUpgradeable.` The combined use of these two patterns is known to create vulnerabilities such as Address Spoofing mentioned by OpenZeppelin. `ERC2771` is designed to securely identify the original sender `(_msgSender())` in meta-transactions via a trusted forwarder. `Multicall` allows batching multiple function calls into a single transaction. When a call to an `ERC2771`-aware function is nested within a `multicall`, the `_msgSender()` context can be manipulated, potentially causing it to return an incorrect address.

Impact

This conflict can lead to address spoofing vulnerabilities, where an attacker could bypass access control checks that rely on `_msgSender()`. By crafting a malicious `multicall` transaction, an attacker might be able to execute privileged actions. This issue is likely present in other `DataPortability` contracts that use the same inheritance pattern.

Recommendation

1. **Decouple Patterns:** The most secure approach is to avoid using both `MulticallUpgradeable and ERC2771ContextUpgradeable` in the same contract. Re-evaluate if both are strictly necessary for the contract's functionality.
2. **Implement Context Isolation:** If both are required, ensure that functions relying on `_msgSender()` from `ERC2771`cannot be called via `multicall` or that the context is carefully managed.

Status

Resolved

Low

[L-01] DataPortabilityServersImplementation#setUserNonce - Nonce Manipulation Allows Signature Replay Attacks

Description

The `setUserNonce` function, restricted to the `MAINTAINER_ROLE`, allows setting a user's nonce to any arbitrary value. This violates the security principle that nonces for signature verification should be strictly and monotonically increasing.

Impact

A malicious or compromised maintainer could lower a user's nonce, making previously used and invalidated signatures valid again. This enables replay attacks, allowing the maintainer to re-execute old, signed operations on behalf of the user, breaking the trust model of signed interactions.

Recommendation

We recommend removing this function entirely or maybe explicitly state in the comments and emit an Event.

Status

Acknowledged

QA

[Q-01] DataPortabilityContracts - Inefficient Storage Access in View Functions Increases Gas Costs

Description

Several `view` functions declare local variables as `storage` pointers to read from state. Accessing struct fields via a storage pointer can result in multiple `SLOAD` opcodes, which is less gas-efficient than loading the entire struct into a `memory` variable once and reading from memory thereafter.

Impact

This pattern leads to higher-than-necessary gas costs for clients and dApps calling these view functions. While it does not introduce a security vulnerability, it deviates from gas optimization best practices.

Recommendation

As recommended it is better to stick to memory for view functions or try using Transient Storages to make it cheaper as compared to Storage.

Status

Resolved

[Q-02] DataPortabilityContracts - Redundant Function Implementations Increase Code Size and Reduce Clarity

Description

The contract contains multiple functions with different names that provide identical functionality. For example, `granteeInfo(uint256 granteeId)` appears to be a duplicate of the public mapping getter `grantees(uint256)`. This redundancy increases deployment costs and maintenance overhead.

Impact

This leads to code bloat, potential confusion for developers integrating with the contract, and slightly increased deployment gas costs.

Recommendation

Consolidate functionality by removing redundant functions. Update all internal and external callers to use a single, canonical function for each piece of logic. If aliases are required for interface compatibility, clearly document their purpose.

Status

Acknowledged

[Q-03] DataPortabilityPermissionsImplementation - Unused Function Parameters

Description

The internal function `_addPermissionFromServerFiles` accepts a `signature` parameter that is never used within the function body.

Unused parameters can confuse developers, suggest incomplete or incorrect logic, and slightly increase calldata costs for no benefit.

Recommendation

Remove the unused `signature` parameter from the function definition to improve code clarity and maintainability. If the parameter is a placeholder for future functionality, add a `// TODO` comment explaining its purpose.

Status

Resolved

Centralisation

The Vana project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Conclusion

After Hashlock's analysis, the Vana project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved or acknowledged . Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au

